

INFRAESTRUCTURAS PARALELAS Y DISTRIBUIDAS-01 – TALLER 3

Carlos Manuel Villamil Grisales – 2257751-3743

Punto 1: Identificación del problema del mundo real

Problema: Monitoreo y evaluación de seguridad en tiempo real para sistemas de telemetría IoT en un entorno industrial.

Justificación: En escenarios como plantas químicas o fábricas automatizadas, existen miles de sensores (en este caso de temperatura y presión) que emiten métricas continuamente. Si un sistema centralizado intentara recibir la lectura de un sensor y, de forma secuencial, evaluar si los valores sobrepasan los niveles críticos, el tiempo de procesamiento bloquearía la recolección de los demás sensores. Esto causaría pérdida de información en tiempo real. Aplicar un enfoque concurrente resuelve el problema separando la tarea de recolectar los datos de la tarea de analizarlos y generar alertas.

Punto 2: Programa creado

<https://github.com/CarlossVillamil/INFRAESTRUCTURAS-PARALELAS-Y-DISTRIBUIDAS-01/tree/main/Taller%203>

Punto 3: Explicación del código, fortalezas y debilidades

Explicación del código

El programa resuelve el problema del Punto 1 con el patrón productor-consumidor visto en clase, en su versión con varios productores y varios consumidores. Tres hilos generan lecturas de sensores y dos hilos las revisan. Entre ellos hay un buffer acotado de 10 posiciones.

Configuración y estado compartido. Las constantes del inicio definen el tamaño del buffer, cuántas lecturas genera cada productor, cuántos hilos hay de cada tipo y los umbrales de alerta (85 °C y 100 psi). Los hilos comparten tres recursos: el buffer `buffer_telemetria`, la consola y el diccionario estadísticas. Cada uno se protege con un mecanismo distinto.

El buffer. `queue.Queue(maxsize=10)` es el buffer acotado del problema clásico y cubre sus dos condiciones de espera. Si el buffer está lleno, `put()` bloquea al productor hasta que se libere un espacio. Si está vacío, `get()` bloquea al consumidor hasta que llegue una lectura. Por dentro, `Queue` usa un lock y variables de condición (`wait` y `notify`), así que el hilo bloqueado duerme en vez de quedarse preguntando en un ciclo (`busy waiting`). Usé `Queue` en lugar de implementar el buffer a mano con `Condition` porque ya resuelve esa sincronización y es la opción que recomienda el material.

Productor. `productor_sensores` arma un diccionario por cada lectura con el id del productor, un id de sensor, un id de lectura y valores aleatorios de temperatura (50 a 100) y presión (70 a 120). Lo mete al buffer con `put()`, imprime el estado y duerme entre 0,05 y 0,2 s para simular el intervalo entre mediciones. Después de 10 lecturas termina.

Consumidor. `consumidor_alertas` repite un ciclo: saca una lectura con `get()`, la compara con los umbrales, imprime "alerta" u "ok" y duerme entre 0,1 y 0,25 s, que representa el tiempo de análisis. Lleva la cuenta de lecturas procesadas y de alertas en variables locales (`procesadas` y `alertas`). Como cada hilo tiene sus propias variables, no necesita lock mientras procesa. Solo al terminar entra una vez a la sección crítica protegida por `lock_stats` para sumar sus parciales al diccionario global, igual que en el ejemplo de clase donde cada hilo calcula un resultado parcial y luego se combinan.

Exclusión mutua con locks. Sin `lock_stats`, la línea `estadisticas["procesadas"] += procesadas` tendría una condición de carrera. La operación lee el valor, lo suma y lo escribe; si dos consumidores la ejecutan intercalados, uno de los dos incrementos se pierde. El bloque `with lock_stats`: garantiza que solo un hilo a la vez ejecute esas líneas, y libera el lock al salir del bloque aunque ocurra una excepción. `lock_print` hace lo mismo con la consola: la función `log()` lo toma antes de cada `print` para que las líneas de distintos hilos no se mezclen.

Cierre del programa. El hilo principal crea y arranca los cinco hilos, y espera con `join()` a que terminen los tres productores. Cuando ya no van a llegar más lecturas, mete en el buffer un `None` por cada consumidor. El consumidor que saca un `None` sale del ciclo. Si cada productor enviara su propio `None` al terminar, un consumidor podría leer la señal de un productor rápido y apagarse mientras los otros siguen produciendo. Por último, la principal espera a los consumidores con `join()` e imprime el resumen.

Resultados de ejecución. En tres ejecuciones el resumen mostró siempre 30 lecturas generadas y 30 procesadas, repartidas entre los dos consumidores (15/15 o 14/16). El programa tardó entre 2,7 y 3,0 s. Hecho en secuencia, el mismo trabajo tomaría unos 9 s: 30 lecturas por los tiempos promedio de generación (0,125 s) y análisis (0,175 s). En la salida, el buffer llega a 10/10 varias veces. Los productores generan más rápido de lo que dos consumidores alcanzan a analizar, y en esos momentos `put()` los detiene.

Fortalezas

- El buffer acotado regula la velocidad de los productores. Cuando los consumidores se atrasan, los productores esperan en lugar de acumular datos sin límite en memoria. Si el atraso es permanente, basta con subir `num_consumidores`, sin cambiar el resto del código.
- La sincronización explícita es mínima. Queue protege los datos en tránsito, y el único estado compartido que se modifica a mano (las estadísticas) se toca una vez por consumidor, no una vez por lectura.
- Los conteos cuadran en todas las ejecuciones: las lecturas procesadas son iguales a las generadas, sin pérdidas ni duplicados.
- El cierre es ordenado. Ningún consumidor termina antes de que se acaben los datos, y ningún hilo queda bloqueado al final.

Debilidades

- No hay tolerancia a fallos. Si un consumidor lanza una excepción, el hilo muere sin registrar sus estadísticas. Si mueren todos, nadie vacía el buffer y los productores quedan bloqueados en `put()` para siempre, porque ni `put()` ni `get()` tienen timeout.
- `qsize()` es aproximado. El productor lee el tamaño después del `put()`, y en ese intervalo otro hilo pudo sacar o meter elementos. El número que se imprime es una referencia aproximada.
- `task_done()` se llama, pero el programa nunca usa `buffer_telemetria.join()`. El cierre depende solo de los `join()` de los hilos y de los centinelas.
- Todas las lecturas tienen la misma prioridad. Una lectura crítica espera en la cola detrás de lecturas normales. Una `PriorityQueue` permitiría atender primero las más graves.
- La simulación es finita, porque cada productor genera 10 lecturas y termina. Un sistema real corre sin parar y necesitaría otra forma de apagado, por ejemplo un `threading.Event`.
- El GIL de Python solo deja que un hilo ejecute bytecode a la vez. Aquí no se nota porque los hilos pasan casi todo el tiempo en `time.sleep`, que libera el GIL. Si el análisis fuera un cálculo pesado, los hilos no aprovecharían varios núcleos y habría que usar `multiprocessing`.
- Todos los hilos pasan por `lock_print` en cada lectura. En esta simulación ese costo es pequeño frente a los `sleep`, pero con mucho volumen de logs la espera por la consola podría volverse un cuello de botella.